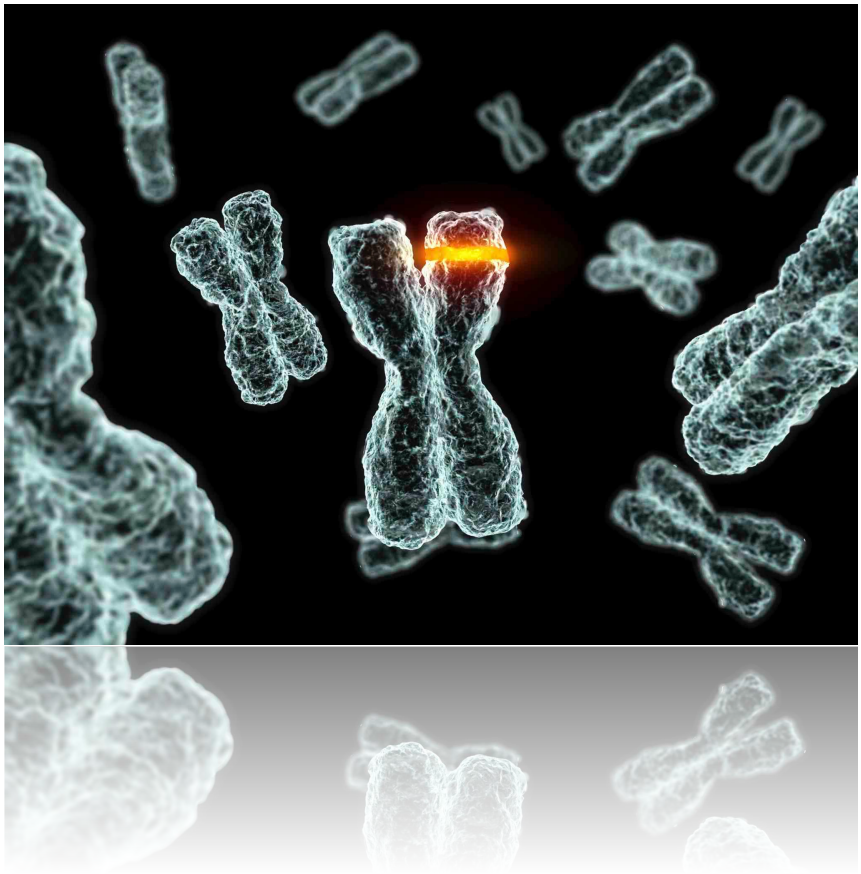
Computational Intelligence II: Evolutionary Optimisation

Chapter 3: Advanced Genetic Search Operators



Fitness transformation

⌘ A selection scheme defines the probability p_i for selecting the i^{th} member c_i of a population to mate. If $F(\Delta[c_i])$ is the **objective value** of that member, and **$f(c_i)$** its fitness, the higher $f(c_i)$ is, the more frequently the member is selected.

$$p_i = f(c_i) / \sum_{k=1}^{N} f(c_k)$$

⌘ The simple case where $f(c_i) = F(\Delta[c_i])$ may not always be appropriate for direct use by the GA, since values may be negative or may not scale the fitness discrepancies between individuals objectively.

⌘ Transforming the fitness appropriately is a critical task as it can cause these two very serious situations:

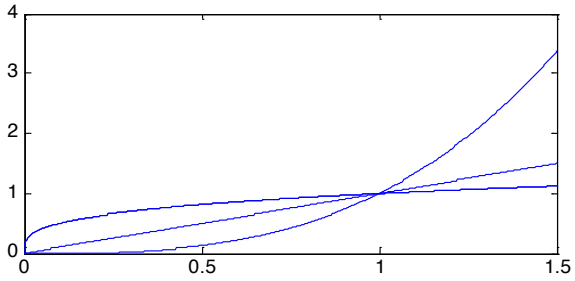
⊙ If **selective pressure** (the frequency of selecting the fitter members) is too small, we have the **Close-Race case**, which refers to cases where the best members and the average ones have similar fitness values; this relaxes competition and makes search a **random walk**.

⊙ If **selective pressure** is high, we have the **Super-Individual case**. This causes **premature convergence**, because a small number of members contribute a disproportionately large number of offspring, since their fitness values are much higher than the rest of the population.

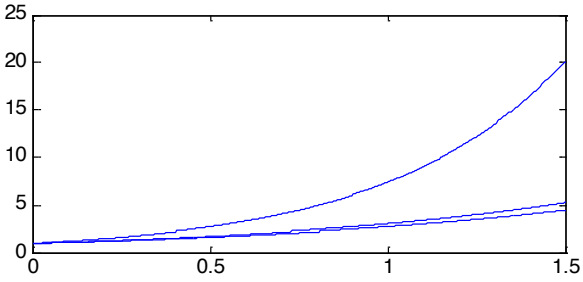
Transformation schemes

⌘ Therefore, a number of transformations for the objective values $F(\Delta[c_i])$ has been proposed:

- **Directly scaled**: to make all fitness values positive.
$$f(c_i) = F(\Delta[c_i]) - F(\Delta[c_{worst}])$$
- **Linear scaling**: where a & b regulate the selective pressure.
$$f(c_i) = a \times F(\Delta[c_i]) + b$$
- **Sigma truncation**: where σ is the standard deviation of the population, μ the mean, $a \in [1,3]$ is a user-defined parameter for the selective pressure and all negative values are clipped to *0*.
$$f(c_i) = \max[0, F(\Delta[c_i]) + (\mu - a\sigma)]$$
- **Power Scaling**: with a>0 used to control the selective pressure.
$$f(c_i) = F(\Delta[c_i])^a$$



- **Exponential Scaling**: with a>0 used to control the selective pressure.
$$f(c_i) = \exp\left(\frac{F[\Delta[c_i]]}{a}\right)$$



Ranking based transformations

- This class of transformations, have the characteristic of disassociating the fitness f from the objective F and controlling the selective pressure directly, without the need for scaling.
  - The better individuals are still selected more frequently, but not so frequently as to dominate the population; this reduces the super-individual case.
  - Also, the close-race case is prevented, as fitness values are heightened when objective values tend to equalise.
  - Ranking works by firstly sorting (ranking) all the individuals according to the objective values F and then, assigning fitness values f based on a user-defined monotonic curve.

- **Baker's Linear Ranking Scheme:**
  - Probabilities are directly assigned using the formula:

$$p_i = \frac{1}{N} \cdot \left(\eta_{max} - (\eta_{max} - \eta_{min}) \cdot \frac{i-1}{N-1} \right)$$

  - The index "i" corresponds to the ranked index (smaller i, better).
  - η_{max} and η_{min} are the expected fitness values of the best and worst members, respectively.
  - The following constraints hold: $\Sigma p_i=1$, $\eta_{min}=2.0-\eta_{max}$ & $\eta_{max} \in [1,2]$, so the user only needs to choose η_{max}.

⌘ Continued…

- η_{max} controls the selective pressure and balances between the exploration and exploitation ability of the GA.
- For η_{max}=1.0, η_{min} is also 1.0 and the selective pressure is nil; this makes the search to be a random walk and the fitness values to be totally ignored (complete close-race).
- For η_{max}=2, we have η_{min}=0 giving thus a maximal selective pressure, which may lead to premature convergence (super-individual).

~ The previous ranking scheme is linear; other nonlinear schemes exist: ~

⌘ **Nonlinear ranking**: this exhibits strong selective pressure (N is the population size).

$$p_i = \frac{N + 1 - i}{\sum_{j=1}^{N} j}$$

⌘ **Geometric ranking**: where c is a normalisation factor to make the probability sum = 1, and η regulates the pressure.

$$p_i = \frac{\eta(1-\eta)^{i-1}}{c}$$

⌘ **Exponential ranking**: where c is a normalisation factor.

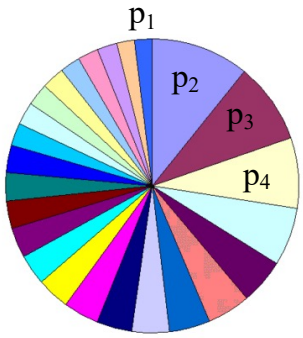
$$p_i = \frac{1 - e^{i}}{c}$$

Selection schemes

⌘ After all fitness values $f(c_i)$ and probabilities p_i have been assigned, probabilistic selection of individuals is repeated in order to find possible parents.

⌘ A standard mechanism is the **roulette-selection** (or **stochastic sampling with replacement**).

$$U = \left[0, \sum_{k=1}^{n} f(c_k)\right]$$



⌘ A uniform random number within U is generated and the first individual i, whose fitness $f(c_i)$ added to all previous ones exceeds this number, is selected (equivalent to spinning the wheel on the right!).

⌘ **Stochastic sampling with no (or partial) replacement** is similar, but it reduces the fitness of an individual each time it is selected, until it is clipped to zero.

⌘ **Stochastic universal sampling** is another selection in which a roulette is spun with a number of markers equalling the population size, positioned at equal intervals around the roulette.

⌘ Continued…

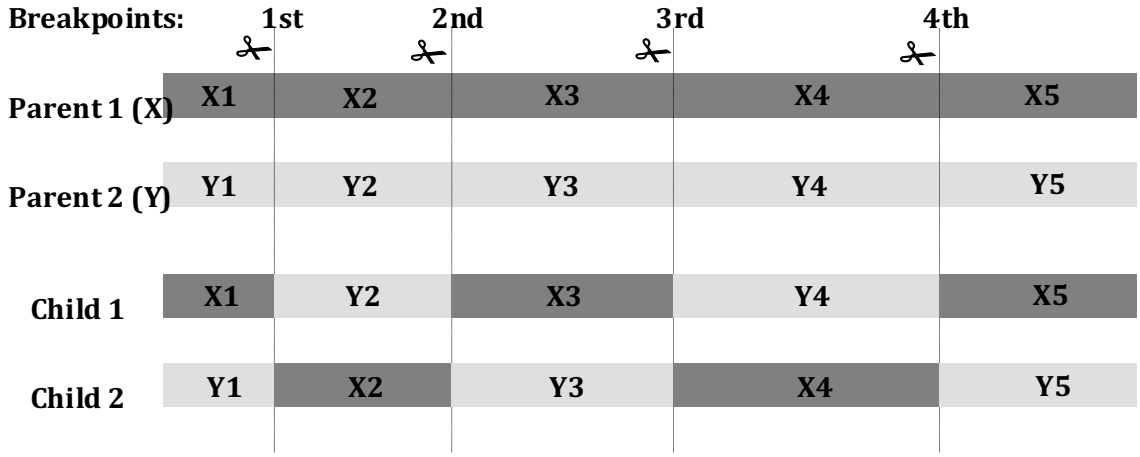
⌘ **k-tournament**: a number of k individuals are picked up and the best one is retained.

⌘ The mathematical analyses of selection mechanisms involve the concept of **Takeover Time** that is the number of generations needed for the fittest individual to dominate the population.

- Short takeover times represent strong selective pressures and emphasise the exploitation aspects of the search.
- Long takeover times emphasise the exploration aspects of a GA and have a weak selective pressure scheme.

Crossover – Discrete types

⌘ **χ -point Crossover:** χ sites are chosen in random in the two parents, and the material is swapped accordingly to produce 2 offspring. Usually we only consider an even χ to treat the chromosome as circular.



◎ For small χ we have **positional bias**, that is a strong tendency in keeping adjacent gene values together and splitting up more distant ones.

◎ For example, Although, a 2-point crossover will keep gene disruption very low, its recombination power will also be low as there are certain chromosomes that the operator cannot combine!

◎ On the other hand, for large χ we have **distributional bias**, which means the tendency to produce children quite dissimilar from the parents.

◎ For example, an 8-point crossover has higher recombination power, but it also causes higher gene disruption.

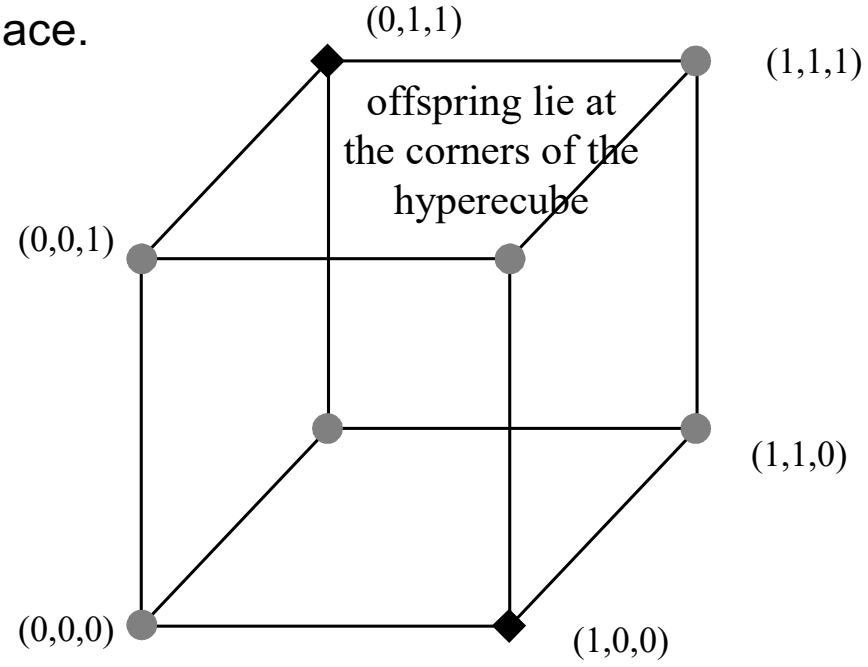
⌘ Continued…

⌘ **Uniform Crossover:** An extreme case of multi-point crossover where alleles are chosen from either parent with a probability of 0.5.

⌘ **Binomial Crossover:** Every allele is taken from the first parent with probability p_r otherwise from the second parent with probability $(1-p_r)$. So, the fraction of alleles taken from the first parent is binomially distributed with mean p_r.

Crossover – Real-valued types

⌘ Very often a real-valued representation is employed, when a continuous function needs to be optimised. In these cases we need additional crossovers, since discrete crossover, excludes a large portion of the continuous search space.



χ-point discrete Xover

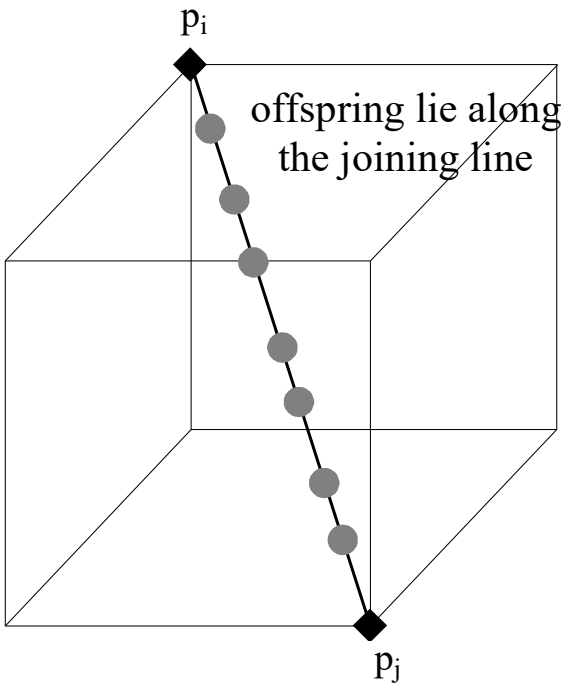
⌘ **Line arithmetical**: A random uniform number $r \in [0,1]$ is generated and then, two offspring c_i & c_j are produced through the following linear combinations of the parental vectors p_i & p_j:

$$c_i = r \times p_i + (1-r) \times p_j$$

$$c_j = (1-r) \times p_i + r \times p_j$$

⌘ Continued…

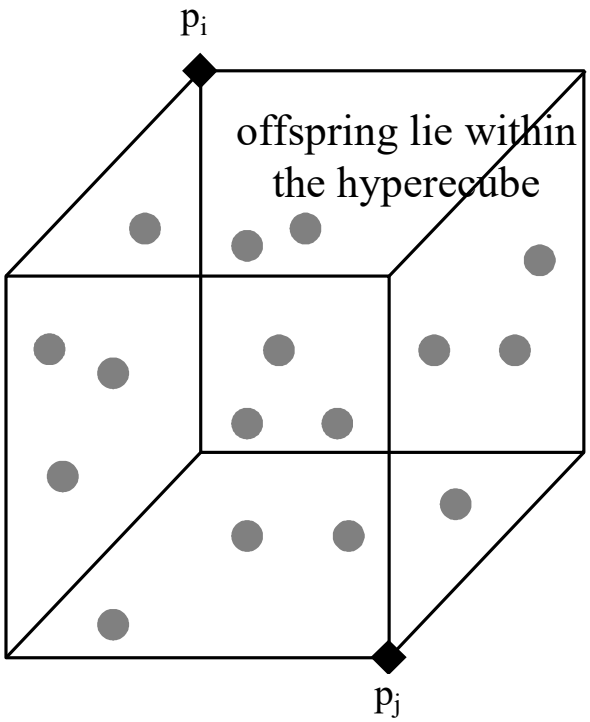
- This crossover produces offspring lying along the line segment joining the two parent points in the solution space.
- Note, that it is a general case of the **Average arithmetical crossover** for fixed r=0.5.



⌘ **Intermediate arithmetical:** It works as the Line arithmetical, but a different r is generated for each gene k.

$$c_i[k] = r_k \times p_i[k] + (1 - r_k) \times p_j[k]$$
$$c_j[k] = (1 - r_k) \times p_i[k] + r_k \times p_j[k]$$

- This crossover generates offspring within the boundaries of the hypercube defined by the two parents.



⌘ Continued…

⌘ **Heuristic arithmetical:** This operator generates only one offspring given by for a random r $\in$ [0,1], where p_i is of better or equal objective value than p_j.

$$c = r \times \left(p_i - p_j \right) + p_i$$

◎ The generated offspring lie at the line passing through the parents, but the comparison of the fitness values biases the operator to search towards the most promising direction.

⌘ **Simplex crossover:** This is a multi-parent crossover, where μ is the mean (centroid) of a random group of parents excluding p_i, with p_i being the worst parent in the group.

$$c = \mu + \gamma \left(\mu - p_i \right)$$

Mutation – Discrete types

⌘ **Bit-flipping.** This is the standard scheme for discrete GAs.

- Mutation flips the 0 or 1 bit to 1 or 0, respectively, with a probability $\mathbf{p_m}$, called the **mutation rate**. This is set to a small number, e.g., 0.01, to avoid disruption of the useful schemata.

- But there is a trade-off. Small p_m can lead to premature convergence due to low exploration capability. Large p_m can lead to randomness and extreme disruption.

- If the encoding is not binary but still based on a discrete alphabet A, e.g., A={a,b,c,d,123, 'qwerty'} then any different value is chosen randomly from A to replace the mutated gene.

- Standard GAs use constant p_m. **Adaptive mutation rates** can be also used to balance efficient exploration and undisrupted exploitation.

- For example, p_m can be adapted by **monitoring the population diversity**. When two parents p_i & p_j mate, their similarity is calculated and the greater the resemblance is, the higher p_m is set. Conversely, when the parents are very dissimilar, there is no need to have high mutation as diversity is obvious. The benefit gained from such adaptation is robustness to diversity loss and premature convergence, caused by small populations and/or high selective pressure.

$$p_m(p_i, p_j) = p_{m_L} + (p_{m_U} - p_{m_L}) \cdot \left[\frac{I(p_i, p_j)}{len} \right]^2$$

- Here, p_{mL} and p_{mU} are the lower and upper mutation bounds set to 0.005 and 0.01, respectively. I(·) is the number of identical genes between the parents p_i & p_j (hamming distance), and len is the length of each parent or child chromosome.

Mutation – Real-valued types

- If we use real-valued representation and assume that each k^{th} gene p[k] can take values from a bounded real alphabet set $A_k = [L_k, U_k]$, then we can use one or more of the following mutation schemes:

- **Uniform mutation.** The new value c[k] is reassigned a random value from its alphabet $A_k = [L_k, U_k]$.

- **Gaussian mutation.** The mutated gene takes values from a Gaussian distribution centred at p[k] with variance σ^2 as:

$$c[k] = N(p[k], \sigma^2)$$

- **Adaptive Non-Uniform mutation**: The value is changed as:

$$c[k] = \begin{cases} p[k] + \Phi(t, U_k - p[k]) \\ p[k] - \Phi(t, p[k] - L_k) \end{cases}, \text{ where } \Phi(t,z) = z \cdot \left(1 - r^{\left(1 - \frac{t}{t_{max}}\right)^b}\right)$$

  - Each branch $\pm$ is chosen with a 0.5 probability (unbiased coin flip).

  - t is the current generation, t_{max} the maximum number of generations, b is a system dependent parameter usually set to 4.0 which reflects the degree of uniformity and $r \in [0,1]$ is a uniform random number.

  - $\Phi(t,z)$ gives a value within [0,z] closer to 0 with higher probability as more generations t elapse. Thus, in the early stages (small t) the space is sampled uniformly, while later on (large t) a local fine-tuning is performed.

Termination of a GA

- As it is not possible to know when the GA will finish since it is a stochastic algorithm, different termination heuristics can be applied (sometimes combinations of them are used):
  - Fix the maximum number of generations t_{max} and stop there.
  - Stop when the difference between the best objective value F_{best} and the average F_{avg} is not more significant that a user-defined threshold ε:

$$\frac{\left|F_{avg} - F_{best}\right|}{\left|F_{avg}\right|} \leq \varepsilon$$

  - Stop if F_{best} has not changed much for the last $t_s = 50$ or so generations:

$$\frac{\left|F_{best,t} - F_{best,t-t_s}\right|}{\left|F_{best,t}\right|} \leq \varepsilon$$

  - Population homogeneity is too high and diversity is too low to carry on (different ways exist to measure homogeneity).
  - Some mathematical termination conditions have been reached, e.g. gradient of the function is near-zero, KKT, error bounds, etc.

Population models

⌘ When a new population of size N is created (N is the population size of the previous generation) and replaces the old population, the model is called **generational** (or **non-overlapping**).

- ⊙ But unfortunate selection may not transmit the best member to the future generations. This can be prevented by the **elitism operator**, which unconditionally copies the fittest member into the next generation.

⌘ When M <N offspring are generated and inserted into the current population, the model is called **overlapping** (or **incremental**).

- ⊙ The ratio M/N is often referred to as the **generation gap**.
- ⊙ For GAs with constant N, M old members have to die. These can be selected to be the worst ones, or selected probabilistically using inverse fitness values.
- ⊙ A robust defence against losing good members, is through the **steady-state model**, which produces only a few individuals per generation, that is M << N.
- ⊙ In a steady-state model, the **no-duplicates** operator is usually employed; this never permits duplicate members within the population at any instance, so that the finite chromosome slots are exploited more productively.

⌘ Continued…

⌘ The **population size** N is a crucial parameter for any GA, and many studies for its optimal setting have been produced.

- In most cases, however, the size is heuristically defined to match the available computational resources for the simulation.
- It is often set to be a multiple of the problem's dimensionality. For example, if the solution **x** is d=8 dimensional, N=10*d could be used.
- It is possible, however, to have GAs with **time-varying population size**. In these models, deletion of old (aged) members is based on their lifetime within the population.

Populations with niching

- For multimodal optimisation it is required to locate all global optima. This is achieved by **Niching**; a method which forms stable subpopulations, each converging to a different peak in parallel. Members within each niche are similar, while between the niches are very different. Niching is very efficient in maintaining diversity.
  - The evaluated fitness values are shared between similar members, depending on the degree of their similarity.
  - This is based on the idea that the within-niche limited resources are shared between the members. Note, that if an individual is on its own in a niche, its fitness is unaffected.
  - Similarity can be based on a Euclidean distance (for real-valued) or Hamming distance (for discrete).
  - A sharing function is usually defined as the form:

$$\gamma\left[d\left(p_i,p_j\right)\right]=\max\left[0,1-\left(\frac{d\left(p_i,p_j\right)}{\sigma_{share}}\right)^a\right]$$

  - where σ_{share} is the sharing radius (or niche radius) within which members are considered similar, and $d(\cdot)$ the distance between the two members p_i & p_j.
  - The new, shared objective values is then defined (for some user-defined b>1) as:

$$F_{shared}\left(p_i\right)=\frac{\left[F\left(p_i\right)\right]^b}{\sum_{j=1}^{N}\gamma\left[d\left(p_i,p_j\right)\right]}$$

Parallel populations

- There are various GA population frameworks that use **parallelism** in different ways:

- **Fine-grained** (or **Cellular**) models use a single population, but individuals are positioned on a grid and genetic operations are performed locally (that is between each member and its immediate neighbours on the grid).

- **Coarse-grained** (or **Island**) models use several independent populations which evolve in parallel.

  - Migration schedules (i.e., moving one member from one population to the other) and inter-population crossovers (i.e., mating between members from different islands) are used to allow some but fully controlled interaction.

  - Such parallel models allow each population to converge independently to different regions of the solution space, so that the chance for premature convergence and thus, sub-optimality, is minimised.

  - Also, they are extremely effective for solving multi-modal problems.

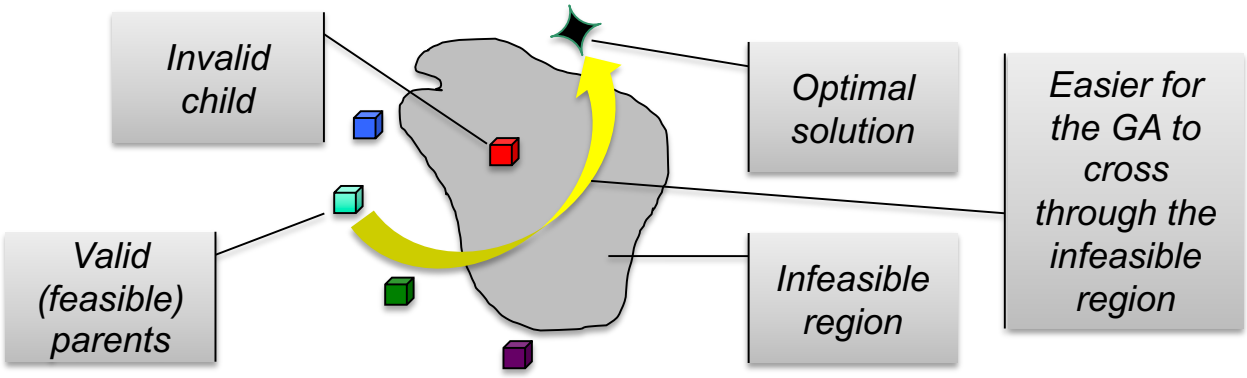
- Simpler types of parallelism involve calculating the objective values of different individuals in parallel.

Hybridisation

- Start with a "no too random" population, that uses good seeds (from other solvers, or human expert's partial knowledge).
- Use intelligent genetic operators especially adapted to the encoding scheme used by the GA.
- Take the best solution(s) finally found by the GA, and then pass them to a local optimisation procedure (e.g., gradient method) for further improvement.
- Perform some local search for each member during evolution (design local search operators – Memetic GAs).
- Use lifetime adaptation using, for example, models such as:
  - Lamarkian model: Traits acquired during the lifetime of an individual can be transmitted to its offspring.
  - Baldwinian model: Traits acquired by an individual cannot be transmitted to its offspring, but its fitness can benefit from adaptation (phenotypic plasticity).

Invalid chromosomes

- Genetic operators can sometimes introduce invalid chromosomes into the population. These are caused either by direct constraints of the optimisation formulation or by the over-expressive power of the encoding, crossover and mutation operators.
- Various resolution mechanisms have been used:
- **Rejection procedure**: Invalid solutions are simply rejected (death penalty).
  - The crossover + mutation operation is then reworked, until a valid child is produced.
  - This is obviously time consuming, especially in cases where it is difficult to generate a valid offspring from some specific parents.
  - Also, in some cases, the search might be more effective if it proceeds through infeasible regions of the search space.



⌘ Continued…

⌘ **Penalty functions**: We can penalise infeasible solutions, depending on the degree of violation.

- The objective function F(·) is modified by adding some terms to measure directly the degree of violation or infeasibility.
- In this way, not only the original objective F(·) is optimised but also the violations are avoided.
- It may be difficult to assign penalties that reflect objectively such violations, but recent studies in time-varying penalty terms have shown high efficiency.
- Example: Assume we have an optimisation problem with m inequalities $g_i(x)$, and n equalities $h_j(x)$.

$$\min_{x \in \Re^d} \quad F(\mathbf{x})$$

$$\text{s.t.} \quad g_i(\mathbf{x}) \le 0, \quad \text{for } i = 1,\ldots,m$$

$$h_j(\mathbf{x}) = 0, \quad \text{for } j = 1,\ldots,n$$

- This can be transformed using user-defined weights a_i, b_j, and user-defined scaling parameters c_i, d_j to:

$$F'(\mathbf{x}) = \underbrace{F(\mathbf{x})}_{\text{original objective}} + \underbrace{\sum_{i=1}^{m} a_i G_i(\mathbf{x}) + \sum_{j=1}^{n} b_j H_j(\mathbf{x})}_{\text{added penalties}}, \quad \text{where:}$$

$$G_i(\mathbf{x}) = \max\left[0, g_i(\mathbf{x})\right]^{c_i}$$

$$H_j(\mathbf{x}) = \left|h_j(\mathbf{x})\right|^{d_j}$$

⌘ Continued…

⌘ **Repair procedure**: Invalid chromosomes are repaired to either replace the invalid ones, or to just be used for evaluation.

- ◎ However, repairing may be very complex and time-consuming (e.g. for Constraint Satisfaction Problems).
- ◎ Also, the repaired version may not be unique or may reside in regions of the search space far away from its invalid version.

⌘ **Design special operators:** Use operators that never produce invalid offspring.

- ◎ Create problem-dependent crossover & mutation operators capable of never producing invalid children.
- ◎ Good examples are the partially-matched (PMX), order (OX) and cycle (CX) reordering crossovers used for order-sensitive combinatorial representations, such as the Travelling Salesman Problem (TSP); see tutorial hand-outs for examples.

⌘ **Decoders**: The chromosome itself contains instructions on how to map it to a valid phenotype. However, designing decoders is complicated as they have to satisfy many conditions.

<END of Chapter 3>